MODULE 27 · EXAM STRATEGY - ALL BLOCKS

Practice paper 3

Thirty new questions, none repeating papers 1 or 2, with four deliberate pairs. The scoring guide reads all three papers together.

QUESTIONS 30 TIMING about 80 s each MODULE 27 of 28

admin@sthithapragna.com

Same rules, and keep all three tallies

- **No paper.** Trace in your head.
- Commit to an answer before each walkthrough.
- Tally your misses by block, next to papers 1 and 2.
- Try to keep to about 80 seconds per question, the exam's pace.

What is the output of this program?

```
1  print(True + True * 3, bool("0"), bool(0.0))
```

A 4 True False

B 6 True False

C 4 False False

D 4 True True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(True + True * 3, bool("0"), bool(0.0))
```

A `4 True False`

True counts as 1: 1 + 1 * 3 is 4. "0" is non-empty; 0.0 is zero.

B `6 True False`

*Adds first: (1 + 1) * 3. Multiplication comes first, so 1 + 3.*

C `4 False False`

"0" is a non-empty string, so it is truthy.

D `4 True True`

0.0 is a zero, so it is falsy.

BLOCK 1

True is 1 in arithmetic. Only empty strings and zeros are falsy.

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for i in range(len(nums)):
3      nums[i] *= i
4  print(nums)
```

A [1, 2, 3]

B [0, 1, 2]

C [0, 2, 6]

D [1, 4, 9]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for i in range(len(nums)):
3      nums[i] *= i
4  print(nums)
```

A `[1, 2, 3]`
*nums[i] *= i changes each slot in the list.*

B `[0, 1, 2]`
Multiplies by i, not replaces with i.

C **`[0, 2, 6]`**
1 * 0, 2 * 1, 3 * 2.

D `[1, 4, 9]`
Each element is multiplied by its index, not itself.

BLOCK 2

Assigning through an index changes the list; assigning to a loop variable does not.

What is the output of this program?

```
1  parts = "a,b,,c".split(",")
2  print(len(parts), "-".join(parts))
```

A 3 a-b-c

B 4 a,b,,c

C 3 a-b--c

D 4 a-b--c

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  parts = "a,b,,c".split(",")
2  print(len(parts), "-".join(parts))
```

A `3 a-b-c`

Two commas in a row leave an empty piece between them.

B `4 a,b,,c`

join uses "-", not a comma.

C `3 a-b--c`

There are 4 pieces, one of them empty.

D `4 a-b--c`

a, b, an empty string, c: joined with -.

BLOCK 3

split(sep) keeps empty pieces; split() with no argument does not.

What is the output of this program?

```
def inner():
    return 1 / 0

def outer():
    try:
        return inner()
    except ZeroDivisionError:
        return "caught"
    finally:
        print("F", end=" ")

print(outer())
```

A caught

B F, then a ZeroDivisionError

C F caught

D caught F

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def inner():
2      return 1 / 0
3
4  def outer():
5      try:
6          return inner()
7      except ZeroDivisionError:
8          return "caught"
9      finally:
10         print("F", end=" ")
11
12 print(outer())
```

A `caught`
finally prints F before the return completes.

B `F, then a ZeroDivisionError`
outer's handler catches what inner raised.

C **`F caught`**
inner's error is caught in outer; finally prints, then caught is returned.

D `caught F`
finally runs before outer actually returns.

BLOCK 4

Errors travel up to the caller's handler. finally runs before the return completes.

> A developer notices a __pycache__ folder containing a file whose name ends in .pyc.

What does a .pyc file in the __pycache__ folder contain?

A The original source code, compressed

B Machine code for the processor

C A copy of the Python interpreter

D Compiled bytecode for an imported module

◆ PAUSE AND DECIDE

What does a .pyc file in the __pycache__ folder contain?

A The original source code, compressed

The source stays in the .py file.

B Machine code for the processor

Bytecode targets the Python virtual machine, not the processor.

C A copy of the Python interpreter

The interpreter is still needed to run it.

D Compiled bytecode for an imported module

Cached so later imports can skip compiling.

BLOCK 1

A .pyc file holds bytecode for an imported module.

What is the output of this program?

```
a, b = 0, 10
while a < b and b > 5:
    a += 2
    b -= 1
print(a, b)
```

A 6 7

B 10 5

C 4 8

D 8 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a, b = 0, 10
2  while a < b and b > 5:
3      a += 2
4      b -= 1
5  print(a, b)
```

A `6 7`

6 < 7 and 7 > 5 is still True: one more pass.

B `10 5`

The loop stops once a < b fails.

C `4 8`

Two passes are not enough to fail the test.

D `8 6`

Passes give (2, 9), (4, 8), (6, 7), (8, 6); then 8 < 6 fails.

BLOCK 2

With and in a loop test, the loop ends as soon as either side is false.

What is the output of this program?

```
1  d = {1: "a", True: "b"}
2  d[1.0] = "c"
3  print(len(d), d[True])
```

A 1 c

B 2 b

C 3 c

D 1 a

◆ PAUSE AND DECIDE

What is the output of this program?

```
d = {1: "a", True: "b"}
d[1.0] = "c"
print(len(d), d[True])
```

A `1 c`

1, True and 1.0 are one key; the last value wins.

B `2 b`

True == 1, so they are the same key.

C `3 c`

Equal keys share one entry.

D `1 a`

Each assignment replaces the value.

BLOCK 3

Equal keys collide: one entry, and the last value assigned wins.

What happens when this program runs?

```
1  count = 0
2  def show():
3      print(count)
4      count = 1
5
6  show()
```

A 0 is printed

B 1 is printed

C Nothing is printed; the program raises a SyntaxError

D The program raises an UnboundLocalError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  count = 0
2  def show():
3      print(count)
4      count = 1
5
6  show()
```

A 0 is printed

count is assigned in show, so it is local throughout show.

B 1 is printed

The print comes before the assignment.

C Nothing is printed; the program raises a SyntaxError

It compiles. It fails when show runs.

D The program raises an UnboundLocalError

count = 1 makes count local, and print reads it first.

BLOCK 4

Paired with paper 2, question 20: assigning anywhere in a function makes the name local everywhere in it.

What happens when this program runs?

```
1  print("start")
2  message = "unfinished
3  print(message)
```

A start, then unfinished

B Nothing is printed; the program raises a SyntaxError

C start, then a SyntaxError

D start, then a NameError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("start")
2  message = "unfinished
3  print(message)
```

A start, then unfinished

The second string is never closed.

B Nothing is printed; the program raises a SyntaxError

An unclosed string is a grammar fault, so line 1 never runs.

C start, then a SyntaxError

Syntax errors stop the whole file, including line 1.

D start, then a NameError

The fault is in the grammar, found before running.

BLOCK 1

Any syntax error, anywhere: nothing runs.

What is the output of this program?

```
x = 4
if x > 2:
    if x > 5:
        print("A")
    else:
        print("B")
print("C")
```

A. A, then C

B. B, then C

C. Only C

D. C, then B

◆ PAUSE AND DECIDE

What is the output of this program?

```
x = 4
if x > 2:
    if x > 5:
        print("A")
    else:
        print("B")
print("C")
```

A. A, then C
4 > 5 is False.

B. B, then C
The else pairs with the inner if, by indentation.

C. Only C
That would need the else to line up with the outer if.

D. C, then B
C prints after the whole if statement.

BLOCK 2

An else belongs to the if it lines up with.

What is the output of this program?

```
1  a = [3, 1, 2]
2  b = sorted(a)
3  a.reverse()
4  print(a, b)
```

A `[1, 2, 3] [1, 2, 3]`

B `[3, 2, 1] [1, 2, 3]`

C `[2, 1, 3] [1, 2, 3]`

D `[2, 1, 3] [3, 2, 1]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [3, 1, 2]
2  b = sorted(a)
3  a.reverse()
4  print(a, b)
```

A `[1, 2, 3] [1, 2, 3]`
reverse() does not sort; and sorted made a separate list.

B `[3, 2, 1] [1, 2, 3]`
reverse() flips the current order. 3, 1, 2 becomes 2, 1, 3.

C `[2, 1, 3] [1, 2, 3]`
sorted copied before the reverse; reverse flips 3, 1, 2.

D `[2, 1, 3] [3, 2, 1]`
b was sorted before a changed, and is a separate list.

BLOCK 3

sorted() makes a new list. reverse() flips the current order; it does not sort.

What happens when this program runs?

```
1  def power(b, e):
2      if e == 0:
3          return 1
4      return b * power(b, e - 2)
5
6  print(power(2, 5))
```

A The program raises a RecursionError

B 32 is printed

C 8 is printed

D 1 is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def power(b, e):
2      if e == 0:
3          return 1
4      return b * power(b, e - 2)
5
6  print(power(2, 5))
```

A The program raises a RecursionError

e goes 5, 3, 1, -1 ... and never equals 0.

B 32 is printed

That needs e - 1, not e - 2.

C 8 is printed

The recursion does not stop at e = 1.

D 1 is printed

The base case is never reached.

BLOCK 4

Check the steps can land exactly on the base case.

What is the output of this program?

```
1  print(str(2.50), int("0042"), float("  1e2 "))
```

A 2.50 42 100.0

B 2.5 0042 100.0

C 2.5 42 100.0

D The code raises a ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(str(2.50), int("0042"), float("  1e2 "))
```

A `2.50 42 100.0`
str() of a float prints its shortest form: 2.5.

B `2.5 0042 100.0`
int() of a str gives a number, so leading zeros vanish.

C `2.5 42 100.0`
str(2.50) is "2.5"; int drops leading zeros; float strips spaces and reads E.

D `The code raises a ValueError`
Each conversion is valid.

BLOCK 1

Spaces around a number string are ignored. Leading zeros in a str are fine for int().

What is the output of this program?

```
count = 0
for c in "Programming":
    if c in "aeiou":
        count += 1
print(count)
```

A 3

B 4

C 2

D 11

◆ PAUSE AND DECIDE

What is the output of this program?

```
count = 0
for c in "Programming":
    if c in "aeiou":
        count += 1
print(count)
```

A 3

o, a and i are the lower-case vowels in Programming.

B 4

There are only three vowels in Programming.

C 2

Counts only o and a; i counts too.

D 11

Only characters in "aeiou" are counted.

BLOCK 2

in on a string checks for a substring, here a single character.

What is the output of this program?

```
1  t = ([1], 2)
2  t[0].append(3)
3  print(t, len(t[0]))
```

A ([1], 2) 1

B ([1, 3], 2) 2

C The code raises a TypeError

D ([1, 3], 2) 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = ([1], 2)
2  t[0].append(3)
3  print(t, len(t[0]))
```

A `([1], 2) 1`
append changes the list inside the tuple.

B `([1, 3], 2) 2`
The tuple holds the same list, which grew.

C `The code raises a TypeError`
No tuple slot is replaced; only the list changes.

D `([1, 3], 2) 3`
len(t[0]) is the length of the inner list: 2.

BLOCK 3

A tuple fixes which objects it holds, not what happens inside them.

What is the output of this program?

```
1  def f(a, b=2, c=3):
2      return a + b * c
3
4  print(f(1, c=10), f(c=1, a=2, b=5))
```

A 10 25

B 21 7

C 21 10

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def f(a, b=2, c=3):
2      return a + b * c
3
4  print(f(1, c=10), f(c=1, a=2, b=5))
```

A `10 25`
b keeps its default, 2, in the first call.

B `21 7`
1 + 2 * 10, then 2 + 5 * 1.

C `21 10`
Keywords match names: a = 2, b = 5, c = 1.

D `The code raises a TypeError`
Every parameter gets exactly one value.

BLOCK 4

Keywords fill by name, in any order. Unfilled parameters use their defaults.

What is the output of this program?

```
1  print(not 1 + 1 == 2, not (1 + 1) == 3)
```

A True False

B False False

C True True

D False True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(not 1 + 1 == 2, not (1 + 1) == 3)
```

A `True False`
not applies last, after the comparison.

B `False False`
1 + 1 == 3 is False, so not gives True.

C `True True`
1 + 1 == 2 is True, so not gives False.

D **`False True`**
not (2 == 2) is False; not (2 == 3) is True.

BLOCK 1

not sits below arithmetic and every comparison.

What is the output of this program?

```
n = 0
while True:
    n += 1
    if n * n > 20:
        break
print(n)
```

A 4

B 20

C 5

D The loop never ends

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 0
2  while True:
3      n += 1
4      if n * n > 20:
5          break
6  print(n)
```

A `4`
*4 * 4 is 16, not more than 20.*

B `20`
n counts up, not to 20.

C `5`
5 * 5 is 25, the first square over 20.

D `The loop never ends`
break fires at n = 5.

BLOCK 2

while True needs a break that is certain to fire.

What is the output of this program?

```
1  s = "Python"
2  print(s[-2] + s[1], chr(ord(s[0]) + 1))
```

A oy P

B ny Q

C hy Q

D oy Q

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "Python"
2  print(s[-2] + s[1], chr(ord(s[0]) + 1))
```

A `oy P`
chr(ord("P") + 1) is the next code point: Q.

B `ny Q`
s[-2] is the second last character, o.

C `hy Q`
s[-2] is o, not h.

D `oy Q`
s[-2] is o, s[1] is y; P shifted by one is Q.

BLOCK 3

-1 is the last character. chr(ord(c) + n) moves n characters along.

What is the output of this program?

```
1 def first_long(words):
2     for w in words:
3         if len(w) > 3:
4             return w
5
6 print(first_long(["hi", "hello", "world"]), first_long(["a"]))
```

A hello world

B hello -1

C hello None

D hi None

◆ PAUSE AND DECIDE

What is the output of this program?

```
1 def first_long(words):
2     for w in words:
3         if len(w) > 3:
4             return w
5
6 print(first_long(["hi", "hello", "world"]), first_long(["a"]))
```

A `hello world`
return ends the function at the first match.

B `hello -1`
Nothing returns -1.

C `hello None`
First long word is returned; no match falls off the end.

D `hi None`
hi has only 2 letters.

BLOCK 4

return ends the function at once. No return reached: None.

What is the output of this program?

```
1  print(hex(255) + bin(3))
```

A `0xff0b11`

B `258`

C `0xff 0b11`

D `The code raises a TypeError`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(hex(255) + bin(3))
```

A `0xff0b11`
hex and bin return strings, so + joins them.

B `258`
The results are strings, not numbers.

C `0xff 0b11`
+ joins with no space.

D `The code raises a TypeError`
str + str is legal.

BLOCK 1

bin(), oct() and hex() return strings.

What happens when this program runs?

```
1  for i in range(3.0):
2      print(i)
```

A 0, 1 and 2 are printed

B 0.0, 1.0 and 2.0 are printed

C Nothing is printed; the program raises a SyntaxError

D The program raises a TypeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  for i in range(3.0):
2      print(i)
```

A 0, 1 and 2 are printed
range() does not accept a float.

B 0.0, 1.0 and 2.0 are printed
range only produces ints, and only accepts them.

C Nothing is printed; the program raises a SyntaxError
The grammar is fine. It fails when range is called.

D The program raises a TypeError
range(3.0): 'float' object cannot be interpreted as an integer.

BLOCK 2

range() takes ints only. A float, even 3.0, is a TypeError.

What is the output of this program?

```
a = ["x"]
a.extend(["yz"])
a += "ab"
print(len(a), a[-1])
```

A 4 b

B 6 b

C 3 ab

D 5 b

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = ["x"]
2  a.extend(["yz"])
3  a += "ab"
4  print(len(a), a[-1])
```

A `4 b`

extend(["yz"]) adds one element; += "ab" adds two characters.

B `6 b`

extend of a list adds that list's single element, "yz".

C `3 ab`

+= with a string adds each character separately.

D `5 b`

"yz" goes in whole: x, yz, a, b.

BLOCK 3

Paired with paper 1, question 13: extend adds each item of what it is given.

What is the output of this program?

```
try:
    print({"a": 1}["b"])
except IndexError:
    print("I")
except LookupError:
    print("L")
except KeyError:
    print("K")
```

A I

B K

C I, then L

D L

◆ PAUSE AND DECIDE

What is the output of this program?

```
try:
    print({"a": 1}["b"])
except IndexError:
    print("I")
except LookupError:
    print("L")
except KeyError:
    print("K")
```

A `I`
A missing key is a KeyError, not an IndexError.

B `K`
LookupError matches first, as KeyError's parent.

C `I, then L`
Only one except branch runs.

D **L**
The KeyError matches its parent, LookupError, first.

BLOCK 4

The first branch that matches the exception or a parent wins.

> Each line is run on its own.

Which two lines print True? (Choose two.)

A `print(0.1 + 0.2 == 0.3)`

B `print(round(0.1 + 0.2, 2) == 0.3)`

C `print(1 == 1.0)`

D `print("1" == 1)`

◆ PAUSE AND DECIDE

Which two lines print True? (Choose two.)

A `print(0.1 + 0.2 == 0.3)`

The sum is 0.30000000000000004.

B `print(round(0.1 + 0.2, 2) == 0.3)`

Rounding removes the tiny error.

C `print(1 == 1.0)`

int and float compare by value.

D `print("1" == 1)`

A str never equals an int.

BLOCK 1

Never compare floats exactly. Numbers compare by value; str never equals int.

What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(i, 3):
        count += 1
print(count)
```

A 6

B 3

C 9

D 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(i, 3):
        count += 1
print(count)
```

A 6

Inner passes: 3, then 2, then 1.

B 3

That was paper 1's range(i). Here it is range(i, 3).

C 9

The inner range shrinks, so do not multiply.

D 4

Adds the passes incorrectly.

BLOCK 2

Paired with paper 1, question 6: when the inner range uses i, count each pass and add.

> d = {"a": 1, "b": 2} has been assigned. Each line is run on its own.

Which two lines remove the key "a" from d? (Choose two.)

A `del d["a"]`

B `d.pop("a")`

C `d.remove("a")`

D `d.discard("a")`

◆ PAUSE AND DECIDE

Which two lines remove the key "a" from d? (Choose two.)

A `del d["a"]`

del removes the pair.

B `d.pop("a")`

pop removes it and returns 1.

C `d.remove("a")`

Dictionaries have no remove method: AttributeError.

D `d.discard("a")`

Dictionaries have no discard method: AttributeError.

BLOCK 3

Dictionaries: del d[k], d.pop(k), d.popitem(), d.clear(). No remove().

What is the output of this program?

```
def g():
    yield 1
    return 2
    yield 3

print(list(g()))
```

A [1]

B [1, 2]

C [1, 3]

D [1, 2, 3]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def g():
2      yield 1
3      return 2
4      yield 3
5
6  print(list(g()))
```

A `[1]`

return ends the generator after the first yield.

B `[1, 2]`

A return value is not yielded.

C `[1, 3]`

Nothing after the return runs.

D `[1, 2, 3]`

return stops the generator.

BLOCK 4

return inside a generator simply ends it.

What is the output of this program?

```
x = 15
if x % 3 == 0:
    print("Fizz", end="")
elif x % 5 == 0:
    print("Buzz", end="")
else:
    print("-", end="")
```

A FizzBuzz

B Fizz

C Buzz

D -

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 15
2  if x % 3 == 0:
3      print("Fizz", end="")
4  elif x % 5 == 0:
5      print("Buzz", end="")
6  else:
7      print("-", end="")
```

A `FizzBuzz`

That was paper 1's answer, with two separate ifs.

B `Fizz`

One chain: the first true branch wins, and Buzz is skipped.

C `Buzz`

The Fizz test comes first and is true.

D `-`

else runs only if both tests fail.

BLOCK 2

Paired with paper 1, question 18: elif joins a chain; a new if starts a new decision.

> Consider Python functions in general.

Which two statements are true? (Choose two.)

A. A parameter's default is evaluated each time the function is called

B. A function can read a global variable without the global keyword

C. Calling a function that contains yield returns a generator object

D. return can appear only once in a function

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A A parameter's default is evaluated each time the function is called

It is evaluated once, when def runs.

B A function can read a global variable without the global keyword

Only assigning needs global.

C Calling a function that contains yield returns a generator object

The body does not run until a value is requested.

D return can appear only once in a function

It can appear many times.

BLOCK 4

Defaults: once. Globals: read freely. yield: a generator. return: as often as needed.

Paper 3, by block

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 9, 13, 17, 21, 25	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 26, 29	6 to 10
3: Data collections	3, 7, 11, 15, 19, 23, 27	11 to 15
4: Functions and exceptions	4, 8, 12, 16, 20, 24, 28, 30	16 to 21

The trend matters more than any single score

ACROSS PAPERS 1 TO 3	WHAT IT MEANS	WHAT TO DO
27 or more on all three	ready	book the exam; take paper 4 a few days before
rising, now 24 or more	the method is working	revisit any block still weak, then paper 4
one block weak in two or more papers	a real gap, not bad luck	rework that block's modules, then module 22
below 21 on paper 3	not ready yet	rework the weakest block, then repeat papers 1 to 3

WHAT MATTERS MOST

Where the misses cluster across papers matters more than the totals. Fix the block that keeps appearing.

admin@sthithapragna.com